COMP0171 Bayesian Deep Learning

Comprehensive Exam Revision Notes

Based on Past Papers (2023–2025) & Course Slides

Auto-generated from course materials

Contents

Bayesian Inference Foundations

Bayes' Rule

Let $\mathcal{D}$ be the data, θ be parameters of interest, and $\mathcal{H}$ be the hypothesis space. From the course slides:

Bayes' Rule

$$p(\theta|\mathcal{D},\mathcal{H}) = \frac{p(\mathcal{D}|\theta,\mathcal{H})\,p(\theta|\mathcal{H})}{p(\mathcal{D}|\mathcal{H})} \quad \Longleftrightarrow \quad \underbrace{p(\theta|\mathcal{D})}_{\text{posterior}} = \frac{\overbrace{p(\mathcal{D}|\theta)}^{\text{likelihood}}\ \overbrace{p(\theta)}^{\text{prior}}}{\underbrace{p(\mathcal{D})}_{\text{evidence}}}$$

The **evidence** (marginal likelihood) is the normalising constant:

$$p(\mathcal{D}) = \int p(\mathcal{D},\theta)\,\mathrm{d}\theta = \int p(\mathcal{D}|\theta)\,p(\theta)\,\mathrm{d}\theta.$$

Maximum Likelihood and MAP Estimation

MLE & MAP

Maximum Likelihood Estimation:

$$\hat{\theta}_{\text{ML}} = \arg\max_{\theta} \sum_{i=1}^{N} \log p(y_i|x_i,\theta).$$

Maximum A Posteriori Estimation:

$$\hat{\theta}_{\text{MAP}} = \arg\max_{\theta} \sum_{i=1}^{N} \log p(y_i|x_i,\theta) + \log p(\theta).$$

With a Gaussian prior $p(\theta) = \mathcal{N}(\theta|0,\alpha^{-1}I)$ and Gaussian likelihood $p(y|x,\theta) = \mathcal{N}(y|f_\theta(x),\beta^{-1})$, MAP becomes:

MAP = Regularised Loss Derivation (2025 Q2c-ii — 3 marks)

Expand the log-prior and log-likelihood, dropping constants w.r.t. θ:

$$\begin{aligned}
\hat{\theta}_{\text{MAP}} &= \arg\max_{\theta} \left\{ \sum_{i=1}^{N} \log p(y_i|x_i,\theta) + \log p(\theta) \right\} \\
&= \arg\max_{\theta} \left\{ \sum_{i=1}^{N} \left[-\frac{\beta}{2}(y_i - f_\theta(x_i))^2 \right] + \left[-\frac{\alpha}{2}\theta^\top\theta \right] \right\} \\
&= \arg\min_{\theta} \left\{ \underbrace{\frac{1}{2}\sum_{i=1}^{N}(y_i - f_\theta(x_i))^2}_{\text{squared-error loss}} + \underbrace{\frac{\alpha}{2\beta}\|\theta\|^2}_{L_2\text{ regulariser}} \right\}.
\end{aligned}$$

The regularisation strength is $\lambda = \alpha/(2\beta)$, set by the ratio of prior precision to noise precision.

Exam Alert: MAP is not Bayesian (2025 Q2c-iii — 5 marks)

MAP estimation yields a *point estimate* $\hat{\theta}_{\text{MAP}}$. Despite using a prior, it is **not Bayesian** for several reasons:

1. **No posterior distribution**: MAP finds the mode of $p(\theta|\mathcal{D})$ but discards the shape of the posterior. It does not maintain a distribution over θ.
2. **No epistemic uncertainty**: predictions $p(y_*|x_*, \hat{\theta}_{\text{MAP}})$ are point predictions with no parameter uncertainty. There is no way to say "I don't know" about unseen regions.
3. **Not reparameterisation-invariant**: the MAP solution changes under nonlinear reparameterisations of θ, whereas Bayesian inference (integrating over the full posterior) is invariant.
4. **No marginal likelihood**: MAP does not provide a model evidence $p(\mathcal{D})$ for model comparison.

A truly Bayesian approach integrates over the posterior:

$$p(y_*|x_*, \mathcal{D}) = \int p(y_*|x_*, \theta)\, p(\theta|\mathcal{D})\, \mathrm{d}\theta.$$

Logistic Regression: MLE and MAP (2024 Q3a–b)

For binary classification with $y_i \in \{0, 1\}$, features $\phi : \mathbb{R}^D \to \mathbb{R}^K$, and $\hat{y}_i = \sigma(\theta^\top \phi(x_i))$ where σ is the logistic sigmoid:

Logistic Regression Objectives

MLE (binary cross-entropy):

$$\hat{\theta}_{\text{ML}} = \arg\max_\theta \sum_{i=1}^{N} \Big[y_i \log \sigma(\theta^\top \phi(x_i)) + (1 - y_i) \log(1 - \sigma(\theta^\top \phi(x_i))) \Big].$$

MAP with Gaussian prior $\theta \sim \mathcal{N}(0, \beta I)$:

$$\hat{\theta}_{\text{MAP}} = \arg\max_\theta \left\{ \sum_{i=1}^{N} [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)] - \frac{1}{2\beta} \|\theta\|^2 \right\}.$$

The MAP objective adds an L_2 penalty, equivalent to logistic regression with weight decay.

Bayesian Linear Regression (Closed-Form Posterior)

Bayesian Linear Regression (Slides: Linear Models / Bayesian Linear Model)

Define the model with features $\phi : \mathbb{R}^D \to \mathbb{R}^H$ and design matrix $\Phi \in \mathbb{R}^{N \times H}$:

$$w \sim \mathcal{N}(0, \alpha^{-1} I_H), \qquad y_i \sim \mathcal{N}(w^\top \phi(x_i),\ \beta^{-1}).$$

In matrix form: $\mathbf{y} \sim \mathcal{N}(\Phi w,\ \beta^{-1} I)$. The posterior is in closed form:

$$p(w|x_{1:N}, y_{1:N}) = \mathcal{N}(w|\mathbf{m}, \mathbf{S}),$$

where

$$\boxed{\mathbf{S} = (\alpha I + \beta\, \Phi^\top \Phi)^{-1}, \qquad \mathbf{m} = \beta\, \mathbf{S}\, \Phi^\top \mathbf{y}.}$$

Note: the posterior mean $\mathbf{m}$ is identical to the ridge regression solution (with $\lambda = \alpha/\beta$).

Posterior predictive for a new input x_*:

$$p(y_*|x_*, \mathcal{D}) = \mathcal{N}\Big(y_* \;\Big|\; \mathbf{m}^\top \phi(x_*),\ \beta^{-1} + \phi(x_*)^\top \mathbf{S}\, \phi(x_*)\Big).$$

Exam Alert: Tested: 2023 Q5, 2024 Q3, 2025 Q2c — 3/3 years

The Bayesian linear model is the foundation for linearised predictions (Section 6.3), Laplace approximations, and the connection to Gaussian processes. Be able to derive the closed-form posterior from the joint Gaussian and show the MAP–ridge equivalence.

Posterior Predictive Distribution

For a new test input x_* with label y_*:

$$p(y_*|x_*, \mathcal{D}) = \int p(y_*|x_*, \theta)\, p(\theta|\mathcal{D})\, \mathrm{d}\theta.$$

This integral is typically intractable for deep networks and requires approximation (VI, MCMC, ensembles, linearisation).

Epistemic vs. Aleatoric Uncertainty

Uncertainty Decomposition (Slides: Uncertainty Quantification)

$$p(y|x, \mathcal{D}) = \int \underbrace{p(y|x, \theta)}_{\text{aleatoric}}\ \underbrace{p(\theta|\mathcal{D})}_{\text{epistemic}}\ \mathrm{d}\theta.$$

- **Aleatoric uncertainty**: $p(y|x, \theta)$ — irreducible noise in the measurement/data generation process. Cannot be reduced by collecting more data.
- **Epistemic uncertainty**: $p(\theta|\mathcal{D})$ — uncertainty about the correct model/parameters given limited data. Reducible with more data.

Exam Alert: Tested every year: 2023 Q1b, 2024 Q2b, 2025 Q1d

Typical scenario: a dog breed classifier has low confidence on rare breeds with few training images. This is *epistemic* uncertainty (limited data for those breeds), which a Bayesian neural network could capture via the posterior $p(\theta|\mathcal{D})$.

Variational Inference and the ELBO

KL Divergence

$$D_{\mathrm{KL}}(q_\lambda(\theta) \| p(\theta|\mathcal{D})) = \mathbb{E}_{q_\lambda(\theta)}\left[\log \frac{q_\lambda(\theta)}{p(\theta|\mathcal{D})}\right] \geq 0.$$

Asymmetry: $D_{\mathrm{KL}}(q\|p)$ is **mode-seeking** (zero-forcing); $D_{\mathrm{KL}}(p\|q)$ is **mass-covering** (zero-avoiding). Variational inference minimises $D_{\mathrm{KL}}(q\|p)$ because computing expectations under $p(\theta|\mathcal{D})$ is intractable.

ELBO Derivation

Evidence Lower Bound — Core Derivation

Starting from the KL divergence:

$$\begin{aligned} D_{\mathrm{KL}}(q_\lambda(\theta)\|p(\theta|\mathcal{D})) &= \mathbb{E}_{q_\lambda(\theta)}\left[\log \frac{q_\lambda(\theta)}{p(\theta|\mathcal{D})}\right] \\ &= \mathbb{E}_{q_\lambda(\theta)}\left[\log \frac{q_\lambda(\theta)\, p(\mathcal{D})}{p(\theta,\mathcal{D})}\right] \\ &= \log p(\mathcal{D}) - \underbrace{\mathbb{E}_{q_\lambda(\theta)}\left[\log \frac{p(\theta,\mathcal{D})}{q_\lambda(\theta)}\right]}_{\text{ELBO, } \mathcal{L}(\lambda,\mathcal{D})}. \end{aligned}$$

Rearranging:

$$\underbrace{\mathbb{E}_{q_\lambda(\theta)}\left[\log \frac{p(\theta,\mathcal{D})}{q_\lambda(\theta)}\right]}_{\text{ELBO, } \mathcal{L}(\lambda,\mathcal{D})} = \log p(\mathcal{D}) - D_{\mathrm{KL}}(q_\lambda(\theta)\|p(\theta|\mathcal{D})) \leq \log p(\mathcal{D}).$$

The ELBO can equivalently be written as:

$$\mathcal{L}(\lambda,\mathcal{D}) = \mathbb{E}_{q_\lambda(\theta)}[\log p(\mathcal{D}|\theta)] - D_{\mathrm{KL}}(q_\lambda(\theta)\|p(\theta)).$$

The first term encourages data fit; the second term penalises deviation from the prior.

VAE ELBO (per-datapoint)

For a latent variable model with latent z, decoder $p_\theta(x|z)$, prior $p(z)$, and encoder $q_\phi(z|x)$:

VAE ELBO (Slides: Deep Generative Models)

$$\log p_\theta(x) \geq \mathcal{L}(x;\phi,\theta) = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)] - D_{\mathrm{KL}}(q_\phi(z|x)\|p(z))\,.$$

Summing over data:

$$\sum_{i=1}^{N} \log p_\theta(x_i) \geq \sum_{i=1}^{N} \left\{\mathbb{E}_{q_\phi(z|x_i)}[\log p_\theta(x_i|z)] - D_{\mathrm{KL}}(q_\phi(z|x_i)\|p(z))\right\}.$$

Exam Alert: Tested 2023 Q4c, 2024 Q5, 2025 Q4a/c

ELBO derivation is a *perennial* exam question. Know how to: (1) start from $D_{\mathrm{KL}}(q\|p)$; (2) isolate $\log p(\mathcal{D})$; (3) re-express as reconstruction + KL penalty; (4) adapt to novel settings (missing data, Bayesian decoder weights, weight subsets).

Variational Inference for BNNs (Weight-Space)

For a BNN with weights θ, approximate $p(\theta|\mathcal{D})$ by $q_\lambda(\theta)$:

$$\mathcal{L}(\lambda,\mathcal{D}) = \mathbb{E}_{q_\lambda(\theta)}\left[\sum_{i=1}^{N} \log p(y_i|x_i,\theta)\right] - D_{\mathrm{KL}}(q_\lambda(\theta)\|p(\theta)).$$

Split weights (2024 Q5): partition $\theta = (\theta_b, \theta_p)$, do VI on θ_b and MAP on θ_p:

$$\mathcal{L}(\lambda, \theta_p) = \mathbb{E}_{q_\lambda(\theta_b)}\left[\sum_{i=1}^{N} \log p(y_i|x_i, \theta_b, \theta_p)\right] - D_{\mathrm{KL}}(q_\lambda(\theta_b)\|p(\theta_b)) + \log p(\theta_p).$$

Monte Carlo Methods

Basic Monte Carlo Estimation

For $\mathbb{E}_{p(x)}[f(x)]$, draw $\hat{x}^{(1)}, \ldots, \hat{x}^{(N)} \sim p(x)$ and compute:

$$\bar{F}_N = \frac{1}{N}\sum_{n=1}^{N} f(\hat{x}^{(n)}).$$

Properties: **consistent** (by SLLN) and **unbiased** ($\mathbb{E}[\bar{F}_N] = \mathbb{E}_p[f(x)]$).

Importance Sampling

Importance Sampling

$$\mathbb{E}_{p(x)}[f(x)] = \mathbb{E}_{q(x)}\left[\frac{p(x)}{q(x)}\, f(x)\right] \approx \frac{1}{S}\sum_{s=1}^{S} \frac{p(x^{(s)})}{q(x^{(s)})}\, f(x^{(s)}), \qquad x^{(s)} \sim q(x).$$

Self-normalised importance sampling (when $p(x) = \gamma(x)/Z$ with unknown Z):

$$\mathbb{E}_{p(x)}[f(x)] \approx \sum_{s=1}^{S} \bar{w}_s\, f(x^{(s)}), \qquad \bar{w}_s = \frac{w_s}{\sum_{s'} w_{s'}}, \quad w_s = \frac{\gamma(x^{(s)})}{q(x^{(s)})}.$$

The self-normalised estimator is *biased* but consistent.

Exam Alert: 2024 Q1c

Self-normalised IS with weights $w(x) \propto p(x)/q(x)$ is **biased** for any finite number of samples and any proposal $q(x)$, because it is a ratio of two estimators.

Designing a Good IS Proposal (2025 Q3a)

A good importance sampling proposal $q(x)$ should:

- Have **heavier tails** than the target $\pi(x)$ to avoid infinite-variance weights.
- **Cover the support** of $\pi(x)\, f(x)$, i.e. $q(x) > 0$ wherever $\pi(x)f(x) \neq 0$.
- Minimise the **variance of the importance weights** $w(x) = \pi(x)/q(x)$.

For a Gaussian proposal $q(x) = \mathcal{N}(x|\mu, \sigma^2)$:

- **Select** μ: match the mode of $\pi(x)$ (e.g. via optimisation: $\mu = \arg\max_x \gamma(x)$).
- **Select** σ: wide enough to cover the mass of π. One criterion: match the curvature at the mode (Laplace-style), or use a slightly wider σ to ensure tail coverage.
- **Avoid**: q that is too narrow (high-variance weights from the tails) or too wide (most samples land in low-probability regions, wasting computation).

Markov Chain Monte Carlo (MCMC)

Metropolis–Hastings Algorithm (Slides: More Monte Carlo / DL Inference)

Given current state $\theta^{(t)}$:

1. Sample candidate $\theta' \sim q(\theta'|\theta^{(t)})$.
2. Compute acceptance ratio:

$$\alpha(\theta', \theta^{(t)}) = \min\left(1,\ \frac{\pi(\theta')\, q(\theta^{(t)}|\theta')}{\pi(\theta^{(t)})\, q(\theta'|\theta^{(t)})}\right).$$

3. With probability α, set $\theta^{(t+1)} = \theta'$; otherwise $\theta^{(t+1)} = \theta^{(t)}$.

If $\pi(x) = \gamma(x)/Z$ with unknown Z, the normalising constants cancel:

$$\alpha = \min\left(1,\ \frac{\gamma(\theta')\, q(\theta^{(t)}|\theta')}{\gamma(\theta^{(t)})\, q(\theta'|\theta^{(t)})}\right).$$

Symmetric proposal $q(\theta'|\theta) = q(\theta|\theta')$ (e.g. Gaussian random walk) simplifies:

$$\alpha = \min\left(1,\ \frac{\pi(\theta')}{\pi(\theta^{(t)})}\right).$$

Detailed balance: $\pi(x)\, T(x \to x') = \pi(x')\, T(x' \to x)$. This is a *sufficient* condition for π to be the stationary distribution, but not *necessary* (balance suffices). [2024 Q1j: this is **False** — detailed balance is sufficient but not necessary.]

Independent Metropolis–Hastings (2025 Q3c–d)

Independent MH

Instead of a random-walk proposal $q(x'|x)$, propose from a **fixed** distribution $q(x') = \mathcal{N}(x'|\mu, \sigma^2)$ that does *not* depend on the current state x.

Acceptance ratio (the proposal is no longer symmetric since $q(x'|x) = q(x')$):

$$\alpha = \min\left(1,\ \frac{\gamma(x')\, q(x)}{\gamma(x)\, q(x')}\right) = \min\left(1,\ \frac{\gamma(x')/q(x')}{\gamma(x)/q(x)}\right).$$

Note: the ratio simplifies to comparing importance weights $w(x) = \gamma(x)/q(x)$.

Optimal q: Setting $q(x) = \pi(x)$ makes $\gamma(x')/q(x') = Z$ for all x', so $\alpha = 1$ always. But this is *circular* — if we could sample from π, we would not need MCMC!

When to prefer each:

- **Independent MH**: good when a close approximation to π is available (e.g. a Laplace/variational approximation). Can make large jumps in state space. Fails in high dimensions (weights become highly variable).
- **Random-walk MH**: more robust in high dimensions; does not require a global approximation. But explores slowly (diffusive behaviour), and step size must be tuned (target $\sim$25% acceptance in high-d).

MCMC for Bayesian Neural Networks: Pros and Cons (2023 Q2b)

MCMC for BNN Posterior

Advantages:

- **Asymptotically exact**: given enough samples and mixing, MCMC provides unbiased posterior expectations — no parametric approximation error (unlike VI/Laplace).
- **No distributional assumptions**: does not restrict the posterior to be Gaussian; can in principle capture multimodality and complex correlations.

Disadvantages:

- **Slow mixing**: BNN weight spaces are very high-dimensional with complex geometry (symmetries, saddle points). Chains may take prohibitively long to explore the posterior.
- **Computational cost**: each MCMC step requires a full or large-batch gradient evaluation; collecting enough independent samples is expensive.
- **Difficult diagnostics**: hard to verify convergence or assess how many samples are sufficient.
- **Poor scalability**: standard MCMC does not easily minibatch (SGLD is an approximate remedy).

Langevin Dynamics and SGLD

Langevin Monte Carlo (Slides: DL Inference)

Langevin proposal:

$$q(\theta'|\theta) = \mathcal{N}\Big(\theta' \;\Big|\; \theta + \eta\,\nabla_\theta \log \pi(\theta),\; 2\eta\, I\Big)\,,$$

equivalently:

$$\theta' = \theta + \eta\,\nabla_\theta \log \pi(\theta) + \sqrt{2\eta}\,\epsilon, \qquad \epsilon \sim \mathcal{N}(0, I).$$

This can be used as an MH proposal. The Langevin proposal is *not* symmetric ($q(\theta'|\theta) \neq q(\theta|\theta')$) because the drift term depends on the current state. The full MH acceptance ratio is:

$$\alpha = \min\left(1,\; \frac{\pi(\theta')\, q(\theta|\theta')}{\pi(\theta)\, q(\theta'|\theta)}\right),$$

where $q(\theta'|\theta) = \mathcal{N}(\theta'|\theta + \eta\nabla \log \pi(\theta),\, 2\eta I)$ and $q(\theta|\theta') = \mathcal{N}(\theta|\theta' + \eta\nabla \log \pi(\theta'),\, 2\eta I)$.
In the special case of an *exact* gradient and $\eta \to 0$, the acceptance ratio approaches 1 (the Langevin diffusion is the continuous-time limit). For finite η, the MH correction is needed to maintain the correct stationary distribution.

Stochastic Gradient Langevin Dynamics (SGLD)

SGLD (Welling & Teh, 2011) **skips the accept/reject step** and uses a noisy gradient estimate:

$$\theta^{(t+1)} = \theta^{(t)} + \eta_t\, \hat{g}(\theta^{(t)}) + \sqrt{2\eta_t}\,\epsilon_t,$$

where $\hat{g}$ is a minibatch estimate of $\nabla_\theta \log \pi(\theta)$. This is justified because as $\eta \to 0$, the acceptance probability tends to 1.
SGLD is **not** an exact sampler for finite step size but becomes exact as $\eta_t \to 0$ with appropriate schedule.

Exam Alert: Tested: 2024 Q2d, 2025 Q1a

SGLD is used in big-data settings: it uses minibatches (only a subset of data per iteration). It is an *inexact* sampler for finite step sizes.

Automatic Differentiation

Forward-mode vs. Reverse-mode Autodiff (Slides: Autodiff)

Forward mode: propagates derivatives forward through the computation graph.

- Cost: one forward pass computes $\partial \text{output}/\partial\theta_j$ for *one* input variable θ_j.
- Efficient when # inputs $\ll$ # outputs.

Reverse mode (backpropagation): propagates derivatives backward.

- Cost: one backward pass computes $\partial L/\partial\theta_j$ for *all* parameters θ_j.
- Efficient when # outputs $\ll$ # inputs (i.e. scalar loss, many parameters — the deep learning setting).
- **Higher memory**: must store intermediate values from the forward pass.

Exam Alert: Tested every year: 2023 Q2a, 2024 Q1d, 2025 Q1e/Q5b

Two reasons to prefer reverse mode for training DL models: (1) scalar loss with millions of parameters $\Rightarrow$ one backward pass suffices; (2) computational cost is $O(1)\times$ forward pass per gradient, vs. $O(P)\times$ for forward mode. Forward mode is better when: few inputs, many outputs (e.g. computing Jacobians of a function $f : \mathbb{R} \to \mathbb{R}^M$, or computing directional derivatives).

Deep Learning Architectures

Fully-Connected Layers

A hidden layer:

$$\mathbf{h} = g(\mathbf{W}\mathbf{x} + \mathbf{b}),$$

where $\mathbf{x} \in \mathbb{R}^D$, $\mathbf{h} \in \mathbb{R}^M$, $\mathbf{W} \in \mathbb{R}^{M\times D}$, $\mathbf{b} \in \mathbb{R}^M$, and $g(\cdot)$ is elementwise.

Number of parameters: $MD + M = M(D+1)$.

Two stacked layers with $\mathbf{h}_1, \mathbf{h}_2 \in \mathbb{R}^M$, $\mathbf{x} \in \mathbb{R}^D$: total parameters $= M(D+1) + M(M+1) = M(D+M+2)$.

Activation Functions (2023 Q3b)

Activation	Advantage	Disadvantage
ReLU: $g(z) = \max(z, 0)$	No vanishing gradient for $z > 0$; sparse	Dead neurons ($z < 0$ gives zero gradient)
Sigmoid: $g(z) = \tanh(z)$	Bounded, smooth	Vanishing gradient for large $\lvert z\rvert$
Identity: $g(z) = z$	Linear, simple	No nonlinearity; stacking adds no capacity

Skip Connections (2023 Q3c, 2025 Q2a)

Adding a skip connection from $\mathbf{x}$ to $\mathbf{h}_2$:

$$\mathbf{h}_1 = g(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1), \qquad \mathbf{h}_2 = g(\mathbf{W}_2\mathbf{h}_1 + \mathbf{V}\mathbf{x} + \mathbf{b}_2),$$

with additional parameter matrix $\mathbf{V} \in \mathbb{R}^{M \times D}$.

Purpose: (1) mitigate vanishing gradients; (2) allow direct gradient flow; (3) make identity mapping easy to learn ("do no harm").

U-Net vs. Autoencoder (2025 Q2a): U-Nets include skip connections to preserve spatial detail for pixel-level tasks (segmentation). Autoencoders *deliberately omit* them because the bottleneck forces a compressed representation; skip connections would bypass the bottleneck.

Convolutional Layers and Equivariance (2024 Q2e, 2025 Q1f)

1D Convolution

For a filter $\mathbf{w} \in \mathbb{R}^3$ and input $\mathbf{x} \in \mathbb{R}^D$, the d-th output element:

$$h_d = \sigma\left(\sum_{k=-1}^{1} w_{k+2}\, x_{d+k}\right) = \sigma(w_1\, x_{d-1} + w_2\, x_d + w_3\, x_{d+1}).$$

Equivariance proof: Define shifted input $x'_{d+1} = x_d$. Then:

$$h'_{d+1} = \sigma(w_1\, x'_d + w_2\, x'_{d+1} + w_3\, x'_{d+2}) = \sigma(w_1\, x_{d-1} + w_2\, x_d + w_3\, x_{d+1}) = h_d.$$

Hence $h'_{d+1} = h_d$: shifting the input shifts the output $\Rightarrow$ **equivariance**.

Note: convolutions are *equivariant* to translation, not *invariant*. Invariance requires additional mechanisms like pooling.

RNNs and Bidirectional RNNs (2024 Q1i, 2025 Q2b)

Standard RNN: processes sequence left-to-right; hidden state $h_t = g(W_h h_{t-1} + W_x x_t + b)$. Can be used for generation (autoregressive).

Bidirectional RNN: two RNNs, one forward, one backward. Hidden state at t depends on the *entire* sequence. Preferred for tasks like classification/tagging where full context is available. *Cannot* be used as a generative model (requires future context).

Monte Carlo Gradient Estimation

We want to estimate:

$$\nabla_\theta\, \mathbb{E}_{p(x|\theta)}[f(x,\theta)].$$

Score Function (REINFORCE) Estimator

Score Function Estimator (Slides: MC Gradient Estimation)

Using the log-derivative trick: $\nabla_\theta p(x|\theta) = p(x|\theta)\nabla_\theta \log p(x|\theta)$,

$$\nabla_\theta\, \mathbb{E}_{p(x|\theta)}[f(x,\theta)] = \mathbb{E}_{p(x|\theta)}[f(x,\theta)\, \nabla_\theta \log p(x|\theta) + \nabla_\theta f(x,\theta)]\,.$$

K-sample estimator:

$$\widehat{\nabla} = \frac{1}{K}\sum_{k=1}^{K}\left[f(x^{(k)},\theta)\, \nabla_\theta \log p(x^{(k)}|\theta) + \nabla_\theta f(x^{(k)},\theta)\right], \quad x^{(k)} \sim p(x|\theta).$$

Applicable to: any differentiable $p(x|\theta)$ (including discrete distributions). Typically has **high variance**.

Reparameterisation (Pathwise) Estimator

Reparameterisation Estimator

Conditions: (1) there exists $g(\theta, \epsilon)$ and $p(\epsilon)$ such that $x = g(\theta, \epsilon)$ with $\epsilon \sim p(\epsilon)$ yields $x \sim p(x|\theta)$; (2) $f(x, \theta)$ is differentiable w.r.t. θ (through g).
Then:

$$\nabla_\theta \, \mathbb{E}_{p(x|\theta)}[f(x, \theta)] = \mathbb{E}_{p(\epsilon)}[\nabla_\theta \, f(g(\theta, \epsilon), \theta)] \, .$$

K-sample estimator:

$$\widehat{\nabla} = \frac{1}{K} \sum_{k=1}^{K} \nabla_\theta \, f(g(\theta, \epsilon^{(k)}), \theta), \quad \epsilon^{(k)} \sim p(\epsilon).$$

Example: for $p(x|\theta) = \mathcal{N}(x|\mu, \sigma^2)$, let $\epsilon \sim \mathcal{N}(0, 1)$ and $x = \mu + \sigma\epsilon$.
Not applicable to: discrete distributions (no differentiable g). Typically **lower variance** than the score function estimator.

Exam Alert: Tested every year: 2023 Q1h, 2024 Q4, 2025 Q5a

Key distinctions: score function works for discrete and continuous; reparameterisation requires continuous, differentiable g and f, but has lower variance. For small discrete support, exact gradient computation may be preferred over MC estimation.

Exact Gradient for Finite Discrete Distributions (2024 Q4c)

If $x \in \{1, 2, 3\}$ with probabilities $\theta_1, \theta_2, \theta_3$, the expectation is a *finite sum*:

$$\mathbb{E}_{p(x|\theta)}[f(x, \theta)] = \sum_{k=1}^{3} \theta_k \, f(k, \theta).$$

The gradient is computed **exactly** by differentiating through the sum — no Monte Carlo estimation needed:

$$\nabla_\theta \, \mathbb{E}_{p(x|\theta)}[f(x, \theta)] = \sum_{k=1}^{3} \nabla_\theta [\theta_k \, f(k, \theta)] = \sum_{k=1}^{3} [f(k, \theta) \, \nabla_\theta \theta_k + \theta_k \, \nabla_\theta f(k, \theta)] \, .$$

This is preferred over the score-function estimator because it has **zero variance**. Use exact computation whenever the support is small and finite.

Bayesian Deep Learning Inference

Posterior Approximation Methods

From the Deep Learning Inference slides, two basic families:

Parametric approximations to $p(\theta|\mathcal{D})$: Gaussian $q(\theta)$ via Variational Bayes or Laplace approximation.

Sample-based approximations: a collection of candidate weights $\theta_1, \ldots, \theta_K$ via MCMC, ensembles, or SGLD.

Laplace Approximation

Laplace Approximation (Slides: DL Inference / Bayesian Linear Models)

Second-order Taylor expansion of $\log p(\theta|\mathcal{D})$ at the mode θ^*:

$$\log p(\theta|\mathcal{D}) \approx \log p(\theta^*|\mathcal{D}) + \frac{1}{2}(\theta-\theta^*)^\top \mathbf{H}\,(\theta-\theta^*),$$

where $\mathbf{H} = \nabla\nabla \log p(\theta|\mathcal{D})\big|_{\theta=\theta^*}$.
This gives a Gaussian approximation:

$$q(\theta) = \mathcal{N}(\theta|\theta^*, \Lambda^{-1}), \qquad \Lambda = -\mathbf{H}.$$

For the regression model with prior $\mathcal{N}(0, \alpha^{-1}I)$ and likelihood variance β^{-1}:

$$\Lambda = \alpha I + \beta \sum_{i=1}^{N} \nabla f_\theta(x_i)\,(\nabla f_\theta(x_i))^\top.$$

(Using the Gauss–Newton / outer-product approximation to the Hessian.)

Advantages: tractable marginal likelihood for hyperparameter optimisation; simple. **Disadvantages**: only captures a single mode; Hessian is expensive.

Linearised Predictions (2023 Q5)

Linearised Laplace Predictive (2023 Q5)

Given $q(\theta|\mathcal{D}) = \mathcal{N}(\theta|\mu, \Sigma)$ and model $y \sim \mathcal{N}(f_\theta(x), \beta^{-1})$:
General predictive (intractable):

$$p(y_0|x_0, \mathcal{D}) = \int \mathcal{N}(y_0|f_\theta(x_0), \beta^{-1})\, q(\theta|\mathcal{D})\, \mathrm{d}\theta.$$

Linearised approximation: Taylor-expand $f_\theta(x_0)$ around $\theta = \mu$:

$$f_\theta(x_0) \approx f_\mu(x_0) + \mathbf{g}(x_0)^\top(\theta-\mu), \quad \text{where } \mathbf{g}(x_0) = \nabla_\theta f_\theta(x_0)\big|_{\theta=\mu}.$$

Since a linear function of a Gaussian is Gaussian:

$$\boxed{p(y_0|x_0, \mathcal{D}) \approx \mathcal{N}\left(y_0 \;\middle|\; f_\mu(x_0),\; \underbrace{\beta^{-1}}_{\text{aleatoric}} + \underbrace{\mathbf{g}(x_0)^\top \Sigma\, \mathbf{g}(x_0)}_{\text{epistemic}}\right).}$$

Assumptions: f_θ is approximately linear in θ near μ (good when posterior is tightly concentrated). Violated for: highly nonlinear networks, flat/broad posteriors, or multimodal posteriors.

This works with *any* Gaussian posterior approximation (Laplace or VB), since the derivation only requires $q(\theta|\mathcal{D}) = \mathcal{N}(\mu, \Sigma)$.

Heteroscedastic Likelihood Extension (2023 Q5c-ii)

If the likelihood variance is **input-dependent**, specified by a second network $\beta_\psi(x)$ with trainable parameters ψ:

$$y_i|x_i \sim \mathcal{N}(f_\theta(x_i),\ \beta_\psi(x_i)^{-1}),$$

then the linearised predictive variance becomes:

$$\mathrm{Var}[y_0|x_0,\mathcal{D}] \approx \underbrace{\beta_\psi(x_0)^{-1}}_{\text{input-dependent aleatoric}} + \underbrace{\mathbf{g}(x_0)^\top \Sigma\, \mathbf{g}(x_0)}_{\text{epistemic}}.$$

Interpretation: the aleatoric term $\beta_\psi(x_0)^{-1}$ now varies with the input — the model can express that some regions of input space are inherently noisier than others (e.g. blurry vs. sharp images). The epistemic term $\mathbf{g}^\top \Sigma\, \mathbf{g}$ still captures parameter uncertainty, which shrinks with more data. This decomposition allows the model to distinguish "I don't know because the data is noisy here" from "I don't know because I haven't seen enough data here."

Deep Ensembles

Deep Ensembles (Tested every year)

Train K models independently from different random initialisations to obtain $\theta_1, \ldots, \theta_K$.
Posterior predictive (MC approximation with uniform weights):

$$p(y_*|x_*,\mathcal{D}) \approx \frac{1}{K}\sum_{k=1}^{K} p(y_*|x_*,\theta_k).$$

For regression: each model predicts $\mathcal{N}(y|\hat{\mu}_k,\hat{\sigma}_k^2)$; the mixture is a mixture of Gaussians.
For binary classification: each model predicts $p(y=1|x,\theta_k)$; the ensemble predictive is $\frac{1}{K}\sum_k p(y=1|x,\theta_k)$, which is a Bernoulli mixture.

Bayesian interpretation: the K parameter values can be viewed as approximate posterior samples from different modes of $p(\theta|\mathcal{D})$. The ensemble average approximates the posterior predictive integral $\int p(y|x,\theta)p(\theta|\mathcal{D})\,\mathrm{d}\theta$.

Why different θ_k? (2025 Q2d-i): Non-convex loss landscape means different initialisations converge to different local optima.

Evaluating $\Pr(y>0|x)$ with an Ensemble (2025 Q2d-ii — 3 marks)

For a regression ensemble with K models, each predicting $p(y|x,\theta_k)=\mathcal{N}(y|\hat{\mu}_k,\hat{\sigma}_k^2)$:

$$\Pr(y>0|x,\mathcal{D}) \approx \frac{1}{K}\sum_{k=1}^{K}\Pr(y>0|x,\theta_k) = \frac{1}{K}\sum_{k=1}^{K}\Phi\left(\frac{\hat{\mu}_k}{\hat{\sigma}_k}\right),$$

where Φ is the standard normal CDF. Each individual model contributes a Gaussian tail probability, and the ensemble averages these.

Alternative Posterior Approximations (2025 Q4g)

Beyond VI, Laplace, and MCMC, other approaches for approximate inference over neural network weights include:

- **SWAG** (Stochastic Weight Averaging–Gaussian): collect weight iterates from SGD with a cyclical/high learning rate, fit a low-rank-plus-diagonal Gaussian to the trajectory. Cheap and easy to implement on top of standard training.
- **Deep ensembles**: treat independently trained models as approximate posterior samples (see above). Not strictly Bayesian but empirically strong.
- **MC Dropout**: use dropout at test time as an approximate variational posterior ($q(\theta)$

is a Bernoulli mask over weights).
- **Laplace post-hoc**: train normally (MAP), then fit a Gaussian around the MAP solution using the Hessian — no change to training, only adds a post-processing step.

Any of these could replace VI for the decoder in a Bayesian VAE (2025 Q4g) or for partial Bayesian inference (2024 Q5).

Importance Sampling and Model Evidence

Estimating the Model Evidence

Two approaches (2024 Q2a):

1. Importance sampling: $p(\mathcal{D}|\mathcal{M}) = \int p(\mathcal{D}|\theta)p(\theta)\,\mathrm{d}\theta = \mathbb{E}_{p(\theta)}[p(\mathcal{D}|\theta)]$, estimable by sampling from the prior.

2. From the ELBO: the ELBO $\mathcal{L} \leq \log p(\mathcal{D})$, so maximising the ELBO gives a lower bound. With a tight ELBO (good q), this approximates $\log p(\mathcal{D})$.

Other approaches: Laplace approximation gives $\log p(\mathcal{D}) \approx \log p(\mathcal{D}|\theta^*) + \log p(\theta^*) + \frac{P}{2}\log(2\pi) - \frac{1}{2}\log|\Lambda|$; harmonic mean estimator from MCMC (but unstable); nested sampling; thermodynamic integration.

Exam Alert: 2024 Q1g, 2025 Q1b

What MCMC can do: compute posterior expectations $\mathbb{E}_{p(\theta|\mathcal{D})}[h(\theta)] \approx \frac{1}{S}\sum_s h(\theta^{(s)})$ by averaging over posterior samples.

What MCMC cannot easily do: estimate the marginal likelihood $p(\mathcal{D})$. MCMC samples from $p(\theta|\mathcal{D})$, but $p(\mathcal{D})$ is the normalising constant that is *cancelled* in the MH acceptance ratio. Naïve estimators (e.g. the harmonic mean $\hat{Z} = [S^{-1}\sum_s p(\mathcal{D}|\theta^{(s)})^{-1}]^{-1}$) have infinite variance and are numerically unstable.

The 2025 T/F statement that MCMC can estimate *both* posterior expectations and marginal likelihoods is therefore **False**.

Active Learning and Bayesian Optimisation

Active Learning (2023 Q1g, 2024 Q2f, 2025 Q1g)

Active learning is appropriate when: labelling is expensive and a model's uncertainty can guide which data points to query next.

BALD — Bayesian Active Learning by Disagreement

Select x^* maximising mutual information between prediction and parameters:

$$I(y;\theta|x^*,\mathcal{D}) = \underbrace{H[y|x^*,\mathcal{D}]}_{\text{total uncertainty}} - \underbrace{\mathbb{E}_{p(\theta|\mathcal{D})}[H[y|x^*,\theta]]}_{\text{expected aleatoric uncertainty}}.$$

This selects points where the model's predictions are most uncertain *due to epistemic uncertainty* (i.e. where individual models "disagree").

Active learning works for any data type (including discrete inputs like emails). The input domain does not need to be real-valued. [2025 Q1g: **False** that active learning requires real-valued inputs.]

Active Learning vs. Bayesian Optimisation (2024 Q2f)

Both iteratively collect data and update a model.

- **Active learning**: goal is to improve the *model* (e.g. classifier accuracy). Selects informative points for labelling.
- **Bayesian optimisation**: goal is to find the *optimum* of an expensive black-box function. Uses acquisition functions (e.g. Expected Improvement, UCB).

Optimisation for Deep Learning

Stochastic Gradient Descent

For an objective $\mathcal{L}(\theta) = \sum_{i=1}^{N} \ell_i(\theta)$, the minibatch gradient estimator using $M \ll N$ points:

$$\hat{g}(\theta) = \frac{N}{M} \sum_{j=1}^{M} \nabla_\theta \, \ell_j(\theta) + \nabla_\theta \log p(\theta).$$

This is an **unbiased** estimate of the full gradient (2024 Q3c–d).

Proof of Unbiasedness (2024 Q3d)

Let $S = \{j_1, \ldots, j_M\}$ be a minibatch drawn uniformly at random from $\{1, \ldots, N\}$. The likelihood part of the estimator is:

$$\hat{g}_{\text{lik}}(\theta) = \frac{N}{M} \sum_{m=1}^{M} \nabla_\theta \, \ell_{j_m}(\theta).$$

Taking the expectation over the random minibatch selection:

$$\mathbb{E}_S[\hat{g}_{\text{lik}}(\theta)] = \frac{N}{M} \sum_{m=1}^{M} \mathbb{E}_{j_m}[\nabla_\theta \, \ell_{j_m}(\theta)] = \frac{N}{M} \cdot M \cdot \frac{1}{N} \sum_{i=1}^{N} \nabla_\theta \, \ell_i(\theta) = \sum_{i=1}^{N} \nabla_\theta \, \ell_i(\theta) = \nabla_\theta \mathcal{L}(\theta),$$

since each j_m is drawn uniformly from $\{1, \ldots, N\}$, so $\mathbb{E}[\nabla_\theta \ell_{j_m}] = \frac{1}{N} \sum_{i=1}^{N} \nabla_\theta \ell_i$.
The prior term $\nabla_\theta \log p(\theta)$ is deterministic and thus unbiased trivially. Therefore $\mathbb{E}[\hat{g}] = \nabla_\theta \mathcal{L}(\theta) + \nabla_\theta \log p(\theta)$, which is the true gradient of the MAP objective.

Warning (Slides: Optimisation): the prior/regularisation term must *not* be rescaled:

$$\hat{\mathcal{L}}_M(\theta) = \log p(\theta) + \frac{N}{M} \sum_{j=1}^{M} \log p(y_j | x_j, \theta).$$

Adaptive Methods

Adam, Adagrad etc. adapt per-parameter learning rates. They still have hyperparameters (initial learning rate, β_1, β_2) that require tuning. [2024 Q1e: **False** that they avoid the need to tune any learning rates.]

Exam Alert: SGD Loss Fluctuation (2024 Q3e)

If the loss increases on a single iteration (e.g. iteration 11 after 10 decreasing), you should *not* stop. SGD uses stochastic minibatch gradients with inherent variance, so loss fluctuations are expected. A single increase does not indicate convergence failure or divergence. **Continue training** and monitor a smoothed (moving-average) loss curve.

Model Comparison

Bayes Factors (Slides: Model Comparison)

Given models $\mathcal{M}_0, \mathcal{M}_1$:

$$\frac{p(\mathcal{M}_0|\mathcal{D})}{p(\mathcal{M}_1|\mathcal{D})} = \underbrace{\frac{p(\mathcal{D}|\mathcal{M}_0)}{p(\mathcal{D}|\mathcal{M}_1)}}_{\text{Bayes factor}} \times \frac{p(\mathcal{M}_0)}{p(\mathcal{M}_1)}.$$

The evidence $p(\mathcal{D}|\mathcal{M}) = \int p(\mathcal{D}|\theta, \mathcal{M}) p(\theta|\mathcal{M})\, \mathrm{d}\theta$ automatically penalises complexity (Occam's razor) because more complex models spread their prior mass more thinly.

Uncertainty Quantification and Calibration

Calibration (Slides: Uncertainty Quantification)

A well-calibrated classifier satisfies:

$$\Pr(\hat{y} = y \mid \hat{p} = p) = p, \qquad \forall p \in [0, 1].$$

Expected Calibration Error (ECE): partition predictions into bins by confidence; ECE measures the weighted average gap between accuracy and confidence.
Temperature scaling: rescale logits by τ before softmax: $\mathrm{softmax}(z/\tau)$. Adjusting τ on a validation set improves calibration without changing predictions.
Modern deep networks tend to be *overconfident*: higher accuracy but worse calibration than older, simpler architectures.

Exam Alert: 2023 Q1f

High accuracy ($> 90\%$) with low confidence ($\sim 60\%$) means the model is **underconfident** / **poorly calibrated**. High accuracy does *not* imply good calibration or good fit.

Deep Generative Models: VAE

VAE Architecture

- **Encoder** (inference network): $q_\phi(z|x)$ maps data to latent space.
- **Decoder** (generative model): $p_\theta(x|z)$ maps latent variables to data.
- **Prior**: $p(z)$, typically $\mathcal{N}(0, I)$.

Training maximises the ELBO (Section 2.3) w.r.t. θ and ϕ jointly, using reparameterised gradients.

Bayesian VAE (2025 Q4)

Introduce a prior $p(\theta)$ over decoder parameters. The joint distribution:

$$p(\theta, z_{1:N}, x_{1:N}) = p(\theta) \prod_{i=1}^{N} p(z_i)\, p_\theta(x_i|z_i).$$

The "fully Bayesian" ELBO targets the log marginal likelihood $\log p(x_{1:N})$ where now θ is also marginalised out.

Fully Bayesian VAE ELBO Derivation (2025 Q4c — 6 marks)

We want a lower bound on $\log p(x_{1:N}) = \log \int p(\theta) \prod_{i=1}^{N} \int p(z_i) p_\theta(x_i|z_i)\, \mathrm{d}z_i\, \mathrm{d}\theta$.
Introduce approximate posteriors $q_\lambda(\theta)$ and $q_\phi(z_i|x_i)$, and define the joint variational distribution:

$$q(\theta, z_{1:N}|x_{1:N}) = q_\lambda(\theta) \prod_{i=1}^{N} q_\phi(z_i|x_i).$$

Apply the standard ELBO construction (KL ≥ 0):

$$\begin{aligned}\log p(x_{1:N}) &\geq \mathbb{E}_{q(\theta, z_{1:N})}\left[\log \frac{p(\theta, z_{1:N}, x_{1:N})}{q(\theta, z_{1:N}|x_{1:N})}\right] \\ &= \mathbb{E}_{q_\lambda(\theta)}\left[\sum_{i=1}^{N} \mathbb{E}_{q_\phi(z_i|x_i)}[\log p_\theta(x_i|z_i)]\right] - \sum_{i=1}^{N} D_{\mathrm{KL}}(q_\phi(z_i|x_i)\|p(z_i)) - D_{\mathrm{KL}}(q_\lambda(\theta)\|p(\theta))\,.\end{aligned}$$

The second line follows by expanding the joint, using the factorisation of q, and grouping terms. The result has three interpretable parts:

1. **Expected reconstruction** under both $q_\lambda(\theta)$ and $q_\phi(z|x)$;
2. **Latent KL**: same as the standard VAE, regularising z_i toward the prior;
3. **Weight KL**: new term $D_{\mathrm{KL}}(q_\lambda(\theta)\|p(\theta))$, regularising decoder weights toward the prior.

Benefit: uncertainty over decoder parameters $\Rightarrow$ better calibrated generative model, less overfitting. Particularly valuable with small datasets or when the decoder is prone to memorisation.

Mini-batch Gradient Estimator (2025 Q4d — 3 marks)

For a minibatch $\{x_1, \ldots, x_M\} \subset \{x_1, \ldots, x_N\}$, the gradient w.r.t. all variational parameters (λ, ϕ) is estimated by rescaling the data-dependent terms:

$$\nabla_{(\lambda,\phi)} \hat{\mathcal{L}}_M = \nabla_{(\lambda,\phi)} \left\{ \frac{N}{M} \sum_{j=1}^{M} \mathbb{E}_{q_\lambda(\theta)}\left[\mathbb{E}_{q_\phi(z_j|x_j)}[\log p_\theta(x_j|z_j)] - D_{\mathrm{KL}}(q_\phi(z_j|x_j)\|p(z_j))\right] - D_{\mathrm{KL}}(q_\lambda(\theta)\|p(\theta)) \right\}.$$

The expectations are estimated via reparameterisation: sample $\theta = h(\lambda, \eta)$ with $\eta \sim p(\eta)$, and $z_j = g(\phi, x_j, \epsilon_j)$ with $\epsilon_j \sim \mathcal{N}(0, I)$. Note the weight-KL term is *not* rescaled by N/M (it appears once, not per-datapoint).

Should we do inference over the encoder? (2025 Q4f)

No, for three reasons:

1. **No generative role**: the encoder $q_\phi(z|x)$ is a variational approximation, not part of the generative model $p(\theta, z, x)$. Its parameters ϕ do not appear in the joint distribution.
2. **Nested variational problem**: placing a prior on ϕ and learning $q(\phi)$ would amount to a *variational approximation to a variational approximation* — conceptually circular and unlikely to improve inference quality.
3. **Amortisation purpose**: ϕ is optimised to minimise the ELBO gap across all datapoints simultaneously. Treating it as a random variable undermines this amortisation, adding computational cost without clear benefit.

Instead, ϕ should be optimised jointly with the Bayesian decoder objective as a standard variational parameter.

Gaussian Processes

Gaussian Process Definition (GP Notes)

A **Gaussian process** is a collection of random variables, any finite subset of which are jointly Gaussian:

$$f \sim \mathcal{GP}(m(\cdot),\ k(\cdot,\cdot)),$$

meaning for any set of inputs $x_1, \ldots, x_N$:

$$\mathbf{f} = [f(x_1), \ldots, f(x_N)]^\top \sim \mathcal{N}(\mathbf{m}, \mathbf{K}),$$

where $m_i = m(x_i)$ and $K_{ij} = k(x_i, x_j)$.

Connection to Bayesian Linear Regression

With features $\phi : \mathbb{R}^D \to \mathbb{R}^H$ and prior $w \sim \mathcal{N}(0, \Sigma)$, define $f(x) = w^\top \phi(x)$. Then:

$$\mathbb{E}[f(x)] = 0, \quad k(x, x') = \phi(x)^\top \Sigma\, \phi(x') = \mathrm{Cov}[f(x), f(x')].$$

A GP with a **linear kernel** $k(x, x') = x^\top \Sigma\, x'$ gives the same predictive distribution as Bayesian linear regression.

Common Kernels

Squared exponential (RBF): $k(x, x') = \exp\left(-\frac{\|x-x'\|^2}{2\ell^2}\right)$. Does not correspond to any finite feature space.

GP Regression Predictive

With observations $\mathbf{y} = [y_1, \ldots, y_N]^\top$, noise variance σ^2, and a new test input x_*:

$$p(y_*|x_*, \mathcal{D}) = \mathcal{N}(y_*|\ \mu_*,\ \sigma_*^2),$$

where

$$\mu_* = \mathbf{k}_*^\top (\mathbf{K} + \sigma^2 I)^{-1} \mathbf{y}, \qquad \sigma_*^2 = k(x_*, x_*) - \mathbf{k}_*^\top (\mathbf{K} + \sigma^2 I)^{-1} \mathbf{k}_* + \sigma^2,$$

and $\mathbf{k}_* = [k(x_*, x_1), \ldots, k(x_*, x_N)]^\top$. The dominant cost is inverting $(\mathbf{K} + \sigma^2 I) \in \mathbb{R}^{N \times N}$: $O(N^3)$ time and $O(N^2)$ memory.

Compare with BLR (Section 1.4): same predictive when k is a linear kernel, but BLR inverts an $H \times H$ matrix ($O(H^3)$), which is cheaper when $H \ll N$.

GP–Neural Network Connection

For a single-hidden-layer network $f_\theta(x)$ with prior $p(\theta)$:

$$m(x) = \mathbb{E}_{p(\theta)}[f_\theta(x)], \quad k(x, x') = \mathbb{E}_{p(\theta)}\left[(f_\theta(x) - m(x))(f_\theta(x') - m(x'))\right].$$

In the infinite-width limit, a BNN with i.i.d. weight priors converges to a GP (Neal, 1995).

Exam Alert: 2025 Q1c: GP with linear kernel

A GP with a linear kernel has the *same* predictive distribution as Bayesian linear regression, but GP regression has cost $O(N^3)$ vs. $O(ND^2 + D^3)$ for the weight-space solution. So it does **not** require less computation. **False**.

Tractable Generative Models and Latent Variable Models

Tractable Generative Models (2023 Q4a)

Examples of models where we can evaluate $p(x)$ directly:

- **Autoregressive models**: $p(x) = \prod_{d=1}^{D} p(x_d|x_{<d})$ — each conditional is parameterised (e.g. by a neural network). Tractable by chain rule.
- **Energy-based models** (with known Z) or **normalising flows**: transform a simple base distribution through invertible maps with tractable Jacobians.
- **Mixture models**: $p(x) = \sum_k \pi_k \, p(x|k)$ with tractable components.

Deep generative models with tractable likelihoods include normalising flows. VAEs do *not* have tractable $p(x)$ (requires marginalising over z).

Generative Models for Inpainting / Missing Data (2023 Q4 — 35 marks)

Let $x = (x^v, x^m)$ be a test image split into visible pixels x^v and missing pixels x^m. We want to estimate $p(x^m|x^v)$.

(a) Tractable model: MCMC for $p(x^m|x^v)$

If $p(x)$ is tractable (e.g. autoregressive model, normalising flow), we can evaluate $p(x)$ for any complete x. Then:

$$p(x^m|x^v) = \frac{p(x^v, x^m)}{p(x^v)} \propto p(x^v, x^m) = p(x).$$

We design a Metropolis–Hastings sampler that only proposes changes to x^m, keeping x^v fixed.

MH for Inpainting with Tractable $p(x)$

Proposal: $q(x^{m\prime}|x^m)$ — e.g. a Gaussian random walk on the missing pixel values: $x^{m\prime} = x^m + \epsilon$, $\epsilon \sim \mathcal{N}(0, \sigma^2 I)$.
Acceptance ratio (symmetric proposal; x^v is fixed throughout):

$$\alpha = \min\left(1, \frac{p(x^v, x^{m\prime})}{p(x^v, x^m)}\right) = \min\left(1, \frac{p(x')}{p(x)}\right),$$

where $x' = (x^v, x^{m\prime})$ and $x = (x^v, x^m)$. The normaliser $p(x^v)$ cancels.
Quality: this is a valid sampler for $p(x^m|x^v)$, but may mix slowly in high dimensions (many missing pixels). Each step only makes small local changes.

(b) VAE: MCMC with latent variables

For a VAE with latent z, the marginal $p_\theta(x) = \int p_\theta(x|z)p(z)\,\mathrm{d}z$ is **intractable**. So we *cannot* directly evaluate $p_\theta(x)$ in the acceptance ratio above.

Solution: extend the MCMC to sample jointly over (x^m, z), targeting $p_\theta(x^m, z|x^v)$:

Joint MCMC for VAE Inpainting (2023 Q4b)

Target: $p_\theta(x^m, z|x^v) \propto p_\theta(x^v, x^m|z)\, p(z)$.
Proposals: alternate or jointly propose:

- For z: use the encoder as a proposal, $q_\phi(z|x^v, x^m)$, or a random walk in z-space.
- For x^m: Gaussian random walk $x^{m\prime} \sim \mathcal{N}(x^m, \sigma^2 I)$, or propose from the decoder $p_\theta(x^m|z)$.

Acceptance ratio (e.g. for a joint proposal):

$$\alpha = \min\left(1,\ \frac{p_\theta(x^v, x^{m\prime}|z')\, p(z')\, q(x^m, z|x^{m\prime}, z')}{p_\theta(x^v, x^m|z)\, p(z)\, q(x^{m\prime}, z'|x^m, z)}\right).$$

All terms are tractable: the decoder likelihood $p_\theta(x|z)$ is known, $p(z)$ is the prior (e.g. $\mathcal{N}(0, I)$), and the proposal q is our design choice.
Does this sample from $p_\theta(x^m|x^v)$? Yes! The joint chain targets $p_\theta(x^m, z|x^v)$, and the x^m marginal of this joint is exactly $p_\theta(x^m|x^v) = \int p_\theta(x^m, z|x^v)\,\mathrm{d}z$. We simply discard the z samples.

(c) Optimising missing pixels: ELBO for $\log p_\theta(x^m|x^v)$

Instead of sampling, we can optimise x^m directly. The key insight is to treat z as a latent variable and derive an ELBO, *exactly as for the standard VAE*:

ELBO for Missing Data (2023 Q4c — 15 marks)

Derivation: start from $\log p_\theta(x^m|x^v)$ and introduce $q(z)$:

$$\begin{aligned}\log p_\theta(x^m|x^v) &= \log \int p_\theta(x^v, x^m|z)\, p(z)\,\mathrm{d}z - \log p_\theta(x^v) \\ &= \log \int \frac{p_\theta(x^v, x^m|z)\, p(z)}{q(z)}\, q(z)\,\mathrm{d}z - \log p_\theta(x^v).\end{aligned}$$

Applying Jensen's inequality on the first term (or equivalently, adding and subtracting $\log q(z)$):

$$\log p_\theta(x^m|x^v) \geq \underbrace{\mathbb{E}_{q(z)}[\log p_\theta(x^v, x^m|z)] - D_{\mathrm{KL}}(q(z)\|p(z))}_{\text{ELBO for } (x^v, x^m)} - \log p_\theta(x^v).$$

Since $p_\theta(x^v)$ is constant w.r.t. x^m, optimising x^m to maximise the ELBO for the full image $\log p_\theta(x)$ is equivalent. Using $q(z) = q_\phi(z|x^v, x^m)$:

$$\boxed{\mathcal{L}(x^m) = \mathbb{E}_{q_\phi(z|x^v, x^m)}[\log p_\theta(x^v, x^m|z)] - D_{\mathrm{KL}}(q_\phi(z|x^v, x^m)\|p(z)).}$$

Maximise $\mathcal{L}(x^m)$ w.r.t. x^m using gradient ascent.
Gradient estimation: use the reparameterisation trick. Sample $\epsilon \sim \mathcal{N}(0, I)$, compute $z = g_\phi(x^v, x^m, \epsilon)$, and differentiate through the decoder and encoder w.r.t. x^m:

$$\nabla_{x^m}\mathcal{L} \approx \frac{1}{K}\sum_{k=1}^{K} \nabla_{x^m}\left[\log p_\theta(x^v, x^m|z^{(k)}) - \log \frac{q_\phi(z^{(k)}|x^v, x^m)}{p(z^{(k)})}\right], \quad z^{(k)} = g_\phi(x^v, x^m, \epsilon^{(k)}).$$

Importance Sampling Gradient Estimator (2023 Q4c-iii)

As an alternative, we can directly estimate $\nabla_{x^m} p(x^m|x^v)$ using likelihood weighting / importance sampling:

$$p(x^m|x^v) = \frac{p(x^v, x^m)}{p(x^v)} = \frac{1}{p(x^v)} \int p_\theta(x^v, x^m|z)\, p(z)\,\mathrm{d}z.$$

Using proposal $q(z) = q_\phi(z|x^v, x^m)$:

$$p(x^m|x^v) \propto \mathbb{E}_{q_\phi(z|x^v,x^m)}\left[\frac{p_\theta(x^v, x^m|z)\, p(z)}{q_\phi(z|x^v, x^m)}\right].$$

Differentiating under the expectation:

$$\nabla_{x^m} p(x^m|x^v) \propto \mathbb{E}_{q_\phi(z|x)}[w(z)\, \nabla_{x^m} \log p_\theta(x^v, x^m|z)]\,,$$

where $w(z) = p_\theta(x|z)p(z)/q_\phi(z|x)$ are importance weights. This can be estimated with self-normalised IS using K samples of z.

Exam Alert: 2023 Q4 — 35 marks, largest single question across all years

This question chains together: tractable generative models, MH-MCMC design, VAE intractability, joint MCMC over latent spaces, ELBO derivation for a novel setting, reparameterised gradients, and importance sampling. Practise the full derivation end-to-end.

Bayesian Last Layer and Partial Bayesian Inference (2024 Q5e)

Bayesian Last Layer

Learn point estimates for all layers except the last output layer, for which we maintain a full posterior.
Let $\theta = (\theta_{\text{feat}}, \theta_{\text{last}})$. Fix $\hat{\theta}_{\text{feat}}$ (via MAP/MLE), then:

$$p(\theta_{\text{last}}|\mathcal{D}, \hat{\theta}_{\text{feat}}) \propto p(\theta_{\text{last}}) \prod_{i=1}^{N} p(y_i|h_i, \theta_{\text{last}}), \quad h_i = f_{\hat{\theta}_{\text{feat}}}(x_i).$$

With features h_i fixed, inference over θ_{last} reduces to Bayesian linear/logistic regression $\Rightarrow$ closed-form (Gaussian) or Laplace posterior.

Advantages: computationally cheap; closed-form posterior for the last layer; leverages deep features. **Disadvantages**: no uncertainty from feature extraction layers; epistemic uncertainty only reflects last-layer ambiguity.

General weight-subset approach (2024 Q5): split $\theta = (\theta_b, \theta_p)$, do VI on θ_b, MAP on θ_p. This generalises the last-layer idea but loses the closed-form advantage; the choice of which weights to be "Bayesian about" is now arbitrary.

Likelihood Factorisation and Minibatching

Likelihood Factorisation (2024 Q1h)

In many models, the likelihood factorises over data points:

$$p(y_{1:N}|x_{1:N}, \theta) = \prod_{i=1}^{N} p(y_i|x_i, \theta).$$

This holds for: linear regression, deep learning classification/regression, and GP regression (conditioned on function values). This factorisation enables **minibatching**: estimating the

full-data gradient using a random subset of $M \ll N$ points:

$$\nabla_\theta \sum_{i=1}^{N} \log p(y_i|x_i,\theta) \approx \frac{N}{M} \sum_{j=1}^{M} \nabla_\theta \log p(y_j|x_j,\theta).$$

Prior scaling warning (Slides: Optimisation): in a MAP/VI objective, only the likelihood is rescaled by N/M; the prior/KL term is *not* rescaled:

$$\hat{\mathcal{L}}_M(\theta) = \log p(\theta) + \frac{N}{M} \sum_{j=1}^{M} \log p(y_j|x_j,\theta).$$

Normalising Constants and Unnormalised Distributions

Working with Unnormalised Distributions (2024 Q1b)

Suppose $p(x|\theta) = \gamma(x,\theta)/Z$ where $Z = \int \gamma(x,\theta)\,\mathrm{d}x$ is unknown.
MLE is still possible if Z does not depend on θ:

$$\nabla_\theta \log p(x|\theta) = \nabla_\theta \log \gamma(x,\theta) - \nabla_\theta \log Z = \nabla_\theta \log \gamma(x,\theta).$$

Even when Z depends on θ, we can sometimes estimate $\nabla_\theta \log Z$ (e.g. via contrastive divergence).
MCMC does not require Z: the MH acceptance ratio uses $\gamma(\theta')/\gamma(\theta)$, and Z cancels.
Importance sampling: self-normalised IS avoids needing Z (but the estimator is biased; see Section 3.2).

Recursive Functions and Automatic Differentiation

Autodiff for Recursive/Stochastic Programs (2025 Q5b)

Consider a recursive function involving random sampling (e.g. the function from 2025 Q5):

```
def f(x):
    z = Uniform(0,1).sample()
    if z > x: return z
    else: return z + f(x)
```

Reverse-mode autodiff: problematic because the computation graph has *variable/unbounded depth* (random recursion depth). The tape must store all intermediate values, but the number of recursive calls is stochastic and potentially infinite.
Symbolic differentiation: not applicable because the function involves stochastic branching and recursion — the closed-form expression for $f(x)$ does not exist as a standard algebraic formula.
Approach via MC gradient estimation: treat $f(x)$ as an expectation. Reformulate as $\mathbb{E}[f(x)]$ over the random choices, then use score-function or other MC gradient estimators to estimate $\frac{d}{dx}\mathbb{E}[f(x)]$ without differentiating through the stochastic control flow.

Solving Inverse Problems with Stochastic Functions (2025 Q5c)

Given a stochastic function $f(x)$ (e.g. the recursive function above), suppose we want to find x such that $f(x) \approx c$ for some target c (e.g. $c = 4$).
Approach: define a loss function over the randomness:

$$L(x) = \mathbb{E}\left[(f(x) - c)^2\right],$$

and minimise $L(x)$ with respect to x via gradient descent. Since autodiff cannot differentiate through the stochastic control flow, we estimate $\nabla_x L(x)$ using the **score-function estimator**. Write $f(x)$ as an expectation over the random variables $z_1, z_2, \ldots$ used in the function. Let $p(z_{1:T}|x)$ denote the joint distribution of all random variables (which depends on x through the branching). Then:

$$\nabla_x L(x) = \nabla_x \, \mathbb{E}_{p(z_{1:T}|x)}\left[(f(x) - c)^2\right].$$

Apply the log-derivative trick:

$$\nabla_x L(x) = \mathbb{E}_{p(z_{1:T}|x)}\left[(f(x) - c)^2 \, \nabla_x \log p(z_{1:T}|x)\right],$$

which can be estimated by running the function K times, recording the random choices, and averaging.

Key Probabilistic Facts for True/False Questions

1. **Zero covariance $\neq$ independence** (2024 Q1a): $\mathrm{Cov}(x, y) = 0$ does not imply $p(x, y) = p(x)p(y)$. Covariance only measures *linear* dependence.
2. **MLE with unknown normaliser** (2024 Q1b): For $p(x|\theta) = \gamma(x, \theta)/Z$, if Z does not depend on θ, it cancels in the gradient $\nabla_\theta \log p(x|\theta) = \nabla_\theta \log \gamma(x, \theta)$. So MLE *is* possible without knowing Z. **False**.
3. **VB vs. Laplace** (2023 Q1a): Both VB with a Gaussian and Laplace produce unimodal Gaussian approximations. VB does *not* capture multiple modes. **False**.
4. **BDL advantages with large data** (2023 Q1c): BDL is most advantageous with *small* datasets (limited data $\Rightarrow$ high epistemic uncertainty). With very large datasets, the posterior concentrates and Bayesian/non-Bayesian converge. **False**.
5. **Gaussian posterior predictive** (2023 Q1e): With a Gaussian approximate posterior $q(\theta) = \mathcal{N}(\theta|\mu, \Sigma)$ and Gaussian likelihood $p(y|x, \theta) = \mathcal{N}(y|f_\theta(x), \beta^{-1})$, the predictive

$$p(y|x, \mathcal{D}) = \int \mathcal{N}(y|f_\theta(x), \beta^{-1}) \, \mathcal{N}(\theta|\mu, \Sigma) \, \mathrm{d}\theta$$

   is an *infinite* mixture of Gaussians — one Gaussian component $\mathcal{N}(y|f_\theta(x), \beta^{-1})$ for each value of θ, weighted by $q(\theta)$. It is *not* a single Gaussian (that would only hold under the linearisation approximation of Section 6.3). **True**.
6. **Reparameterisation conditions** (2023 Q1h): The exam states two conditions: (i) f is differentiable w.r.t. θ, and (ii) a reparameterisation $x = g(\theta, \epsilon)$ exists. The first condition is *incomplete*: the reparameterised gradient $\nabla_\theta f(g(\theta, \epsilon), \theta)$ requires f to be differentiable w.r.t. its *first argument* x (so that $\nabla_\theta g$ can propagate through f), not merely w.r.t. θ directly. **False**.
7. **Laplace and decision boundary** (2024 Q1f): The Laplace approximation is centred at the MLE/MAP θ^*, so the predictive mean at the decision boundary is unchanged. The decision boundary is preserved. **True**.
8. **GP with linear kernel** (2025 Q1c): A GP with a linear kernel has the same predictive distribution as Bayesian linear regression, but GP computation scales as $O(N^3)$ which is *worse*

for large N than the $O(ND^2)$ cost of Bayesian linear regression. **False** (same predictive, but not less computation).

9. **Boy/girl conditional probability** (2023 Q1d): A friend has two children; at least one is a boy. She shows you a picture of a son *first*. The probability the other child is also a boy is $\frac{1}{2}$, *not* $\frac{1}{3}$. The sequential reveal changes the conditioning: once a specific child is identified as a boy, the other child's sex is independent. (Compare with the classic "at least one boy" puzzle, where Pr(both boys | at least one boy) = 1/3.) **True**.
10. **Self-driving car uncertainty examples** (2024 Q2b): For a segmentation model trained on UK city data:
    - *Aleatoric* (irreducible noise): nighttime scenes with poor lighting, motion blur, rain/fog obscuring the scene, reflective surfaces. These are noisy even with a perfect model.
    - *Epistemic* (limited training data): rural roads, snow-covered scenes, unusual vehicles (tractors, horse-drawn carts), countries with different road markings. The model has never seen these ⇒ high parameter uncertainty, reducible with more data.
11. **SGD loss going up is normal** (2024 Q3e): If the loss increases on one iteration after 10 decreasing iterations, you should *not* stop. SGD uses stochastic minibatch gradients whose variance causes fluctuations. A single increase does not indicate convergence or divergence. Continue training; monitor a moving average of the loss instead.
12. **Convolution: equivariant ≠ invariant** (2025 Q1f): Convolutional layers are *equivariant* to translation (shifting the input shifts the output correspondingly), **not** invariant (invariance means the output is unchanged by shifting). Invariance requires additional mechanisms such as global average pooling. **False** (the exam claims "invariant").
13. **Active learning works for discrete inputs** (2025 Q1g): The exam claims active learning is impossible for discrete data (e.g. emails). This is **False**: active learning selects which inputs to query for labels based on model uncertainty. There is no requirement that inputs be real-valued. BALD, uncertainty sampling, etc. all apply to discrete or mixed-type inputs.
14. **Bidirectional RNNs cannot generate** (2024 Q1i): Bi-RNNs process a sequence in both directions, so the hidden state at time t depends on future tokens. This makes them unsuitable as autoregressive generative models (generation requires producing tokens sequentially, left-to-right). They *can* be used as feature extractors for classification/tagging. **True**.
15. **SGLD is inexact but useful for big data** (2025 Q1a): SGLD uses minibatch gradients and skips the MH accept/reject step. It is an *inexact* sampler for any finite step size, but is used in big-data settings because each iteration only touches a data subset. **True**.

Exam Question Patterns: Quick Reference

Topic	Years Tested	Typical Question Type
ELBO derivation	2023, 2024, 2025	Derive ELBO from scratch; adapt to new settings
Gradient estimators (score fn / reparam)	2023, 2024, 2025	Write K-sample estimator; compare approaches
MCMC / Metropolis–Hastings	2023, 2024, 2025	Write acceptance ratio; design proposals
Epistemic vs. aleatoric	2023, 2024, 2025	Identify from scenario; explain difference
Deep ensembles	2023, 2024, 2025	Describe predictive; justify as Bayesian
Autodiff (fwd vs. rev)	2023, 2024, 2025	Advantages; when to prefer each
Active learning	2023, 2024, 2025	T/F; when appropriate
Importance sampling	2023, 2024, 2025	Write estimator; bias properties
MLE / MAP	2024, 2025	Write objective; show equivalence to regularisation
Linearised predictions	2023	Full derivation; interpret variance terms
Laplace approximation	2023, 2024	Describe; compute Hessian; properties
VAE / generative models	2023, 2025	ELBO for VAE; extend to Bayesian decoder
Langevin / SGLD	2024, 2025	Write proposal; explain SGLD validity
Convolutional equivariance	2024, 2025	Write 1D conv; prove equivariance
RNN / bidirectional RNN	2024, 2025	Compare; when to use each
Model evidence	2024, 2025	Estimation methods; MCMC limitations
Bayesian linear regression	2023, 2024, 2025	Closed-form posterior; connection to ridge
Gaussian processes	2025	Definition; kernel; GP–NN connection
Tractable generative models	2023	Name examples; contrast with VAE
Bayesian last layer	2024	Advantages; contrast with full VI
Likelihood factorisation	2024	T/F; minibatch gradient scaling
Normalising constants	2024	MLE without Z; MCMC cancellation
Recursive functions + autodiff	2025	Limitations of autodiff; MC gradient approach
Topics added from gap analysis (previously missing):		
Inpainting / missing data	2023	MH for $p(x^m\|x^v)$; joint MCMC over (x^m, z); ELBO for conditional; IS gradient **(35 marks!)**
Independent MH	2025	Acceptance ratio; $q = \pi$ paradox; compare with random walk
Heteroscedastic likelihood	2023	Input-dependent $\beta_\psi(x)$; changes aleatoric term
Logistic regression MLE/MAP	2024	Binary cross-entropy; L_2 regularised logistic regression
IS proposal design	2025	Good proposal criteria; selecting μ, σ
MCMC pros/cons for BNNs	2023	Exact but slow; high-dim mixing; diagnostics
Discrete exact gradient	2024	Finite support $\Rightarrow$ sum, no MC needed
SGD loss fluctuation	2024	Non-monotone loss is normal; don't stop early
Alternative posteriors (SWAG etc.)	2025	SWAG, MC Dropout, post-hoc Laplace